SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE: CSA0606

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 10 **Total Marks: 100**

CO2-AT3 DEBUGGING & OPTIMIZATION REPORT

Code of Conduct:

*I **Guru Hemanth Reddy(192311222)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Hemanth Reddy

1 1. Introduction and Problem Statement

Question: Q40. Debugging Insertion Sort Code

Insertion Sort is a fundamental brute-force sorting algorithm that builds the final sorted array one item at a time. It behaves much like sorting playing cards in your hands: you pick a card and insert it into its correct position within the already sorted segment of cards.

The provided Python code attempts to implement Insertion Sort but contains a critical indexing flaw during the final insertion step. This report carefully analyzes the element-shifting process, identifies the exact index logic error, performs a dry run to demonstrate the failure, and provides a corrected, optimized, and fully commented implementation alongside a comprehensive complexity analysis.

2 2. Code Analysis and Bug Identification

2.1 2.1 The Buggy Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1

        arr[j] = key    # ERROR: Incorrect Placement Index

    return arr
```

2.2 2.2 Identification of the Exact Source of Error

The error resides on line 10: `arr[j] = key`. During the `while` loop, the algorithm shifts elements that are strictly greater than the `key` exactly one position to the right (`arr[j+1] = arr[j]`). After each shift, the index `j` is decremented (`j -= 1`).

When the `while` loop finally terminates, it is because one of two conditions was met: 1. **`j < 0`:** We have reached the very beginning of the array, meaning the `key` is the smallest element encountered so far. 2. **`arr[j] <= key`:** We have found an element that is strictly smaller than or equal to the `key`.

In both termination cases, the index `j` currently points to the slot *before* the correct insertion slot. Therefore, the correct position to insert the `key` is always exactly `j + 1`.

By incorrectly placing the key at `arr[j]`, the buggy code overwrites a correctly sorted element (if `arr[j] <= key`) or attempts to write to `arr[-1]`. In Python, negative indexing targets the end of the array, meaning `arr[-1]` modifies the very last element of the dataset, completely destroying the data's integrity.

3 3. Step-by-Step Debugging and Dry Run

To illustrate the failure and the subsequent correction, let us trace a sample array: `arr = [4, 3, 2]`.

3.0.1 Executing the Buggy Code:

- **Initial Array:** `[4, 3, 2]`
- **Iteration 1 ($i = 1$):**
  - `key = 3`, `j = 0`
  - Condition: `arr[0]` (4) > 3? Yes.
  - Shift: `arr[1] = 4`. Array is now `[4, 4, 2]`.
  - `j` becomes `-1`.
  - Loop terminates.
  - **Bug Execution:** `arr[-1] = 3`. In Python, `arr[-1]` targets the last element. The array corrupts and becomes `[4, 4, 3]`.
- **Iteration 2 ($i = 2$):**
  - `key = 3` (Since `arr[2]` is now 3 due to the bug!), `j = 1`.
  - The array corrupts further and sorting completely fails.

3.0.2 Corrected Logic Trace (`arr[j + 1] = key`):

- **Initial Array:** `[4, 3, 2]`
- **Iteration 1 ($i = 1$):**
  - `key = 3`, `j = 0`
  - Condition: `arr[0]` (4) > 3? Yes. Shift: `arr[1] = 4`. Array: `[4, 4, 2]`.
  - `j` becomes `-1`. Loop terminates.
  - **Correct Execution:** `arr[j + 1]` → `arr[-1 + 1]` → `arr[0] = 3`.
  - Array becomes `[3, 4, 2]`. The prefix is correctly sorted.

4 4. Corrected and Optimized Implementation

Below is the corrected code. It has been optimized for readability with clean, descriptive variable names and explicit structural comments, adhering strictly to Python best practices.

```
def insertion_sort(arr):
    """
```

```
    Sorts a list in ascending order using the Insertion Sort
algorithm.
    """
    # Traverse from the second element to the end of the array
    for current_idx in range(1, len(arr)):
        key_value = arr[current_idx]

        # Initialize the comparison index to the element just
before the key
        compare_idx = current_idx - 1

        # Shift elements of the sorted prefix that are greater
than the key
        # to one position ahead of their current position
        while compare_idx >= 0 and arr[compare_idx] > key_value:
            arr[compare_idx + 1] = arr[compare_idx]
            compare_idx -= 1

        # Correct Placement: Insert the key at the exact slot
opened by the shift
        arr[compare_idx + 1] = key_value

    return arr
```

5 5. Time and Space Complexity Analysis

5.1 5.1 Time Complexity

- **Best Case ($O(n)$):** If the array is already sorted in ascending order, the inner `while` loop condition (`arr[compare_idx] > key_value`) fails immediately on every single iteration. The algorithm simply verifies the order, taking linear time.
- **Worst Case ($O(n^2)$):** If the array is sorted in descending (reverse) order, every single element must be shifted all the way to the beginning. The outer loop runs $n-1$ times, and the inner loop runs $1,2,3,\ldots,n-1$ times. The sum of the first n integers is $\frac{n(n-1)}{2}$, resulting in a quadratic time complexity limit.
- **Average Case ($O(n^2)$):** For a randomly ordered array, an element will be shifted half the distance of the sorted prefix on average, maintaining an $O(n^2)$ computational overhead.

5.2 5.2 Space Complexity

- **Space Complexity ($O(1)$ auxiliary):** Insertion Sort is an *in-place* sorting algorithm. It only requires a single extra memory space to hold the `key_value` variable during the shifting phase. It does not scale its memory footprint with the size of n, making it highly memory-efficient.

6 6. Conclusion

The initial code failed due to a fundamental off-by-one indexing error (`arr[j] = key`). Because the `while` loop decrements `j` *after* performing a shift, the final correct slot for the extracted key is strictly `j + 1`. In languages like Python, negative indexing hides this out-of-bounds error by silently modifying the end of the array, leading to completely unpredictable and destructive outputs. Correcting the line to `arr[j + 1] = key` restores the mathematical integrity of the algorithm, ensuring that elements are properly inserted into the sorted prefix. While carrying an $O(n^2)$ complexity overhead, Insertion Sort remains practically significant for very small datasets or arrays that are already substantially sorted.